SEKOSA: Supplementary Material

A. Component Ontology Models

This section provides additional details on the ontology models used to represent system components in SEKOSA. All component models follow the general structure introduced in the main paper, where a component is subject to a set of requirements regarding its input Creations that need to be realized for it to produce a Creation.

Sensor

A Sensor converts a physical phenomenon in the environment into data. Its environmental requirements (ER) specify the conditions under which the sensed phenomenon can be observed. Additionally, non-functional requirements (NFR) can capture dependencies on available resources. When these requirements are satisfied, the sensor produces data with associated quality attributes. Typical examples include cameras or microphones, where environmental conditions (e.g., light intensity or noise level) directly influence the quality of the produced data.

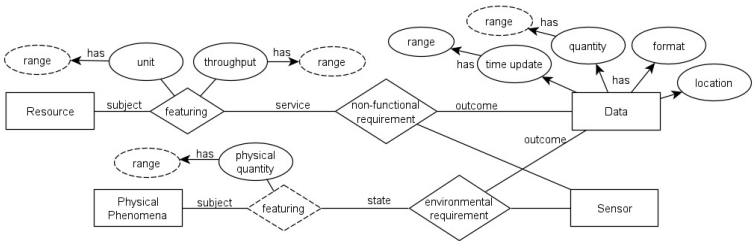

Fig. 1: SEKOSA model of a sensor component.

Actuator

An Actuator transforms data into a physical phenomenon in the environment. Its functional requirements (FR) specify the required input data, while its NFR define required resources such as power, and ER describe required physical phenomena. The output of an actuator is a physical effect in the environment, with associated physical quantities. Typical examples include flashlights or speakers.

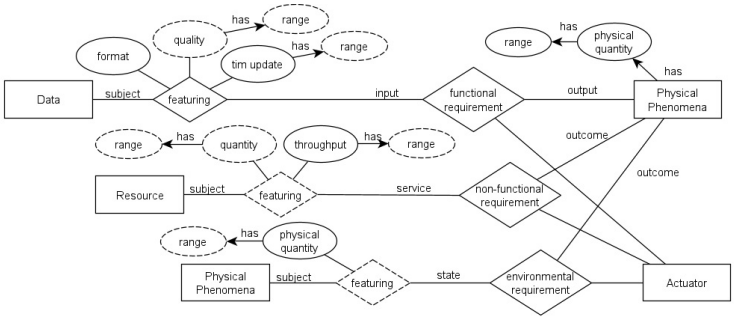

Fig. 2: SEKOSA model of an actuator component.

Functional Component

A Functional component processes input data to produce new data. Like for the actuator, FR describe requirements on input data, NFR on available resources, and ER on environmental conditions. Functional components represent computational processes such as perception, estimation, or decision-making modules.

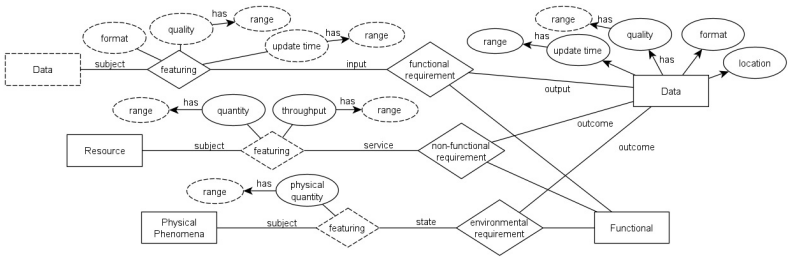

Fig. 3: SEKOSA model of a functional component.

Appliance

An Appliance provides resources required by other components. It does not process input data (thus not having FR) but produces a resource that can be consumed by other components via non-functional requirements. To do so, it may require resources itself (captured by NFR) or certain environmental conditions (captured by ER). Examples include power supplies or computational units. The provided resource is characterized by properties such as capacity, throughput, or quantity.

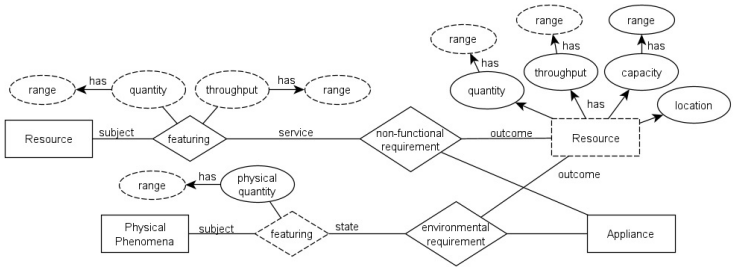

Fig. 4: SEKOSA model of an appliance component.

B. Inference of Realizing and Processing Relations

This section provides additional details on the inference mechanisms underlying SEKOSA, specifically regarding the Realizing and Processing relations inferred based on component requirements and environmental conditions.

Realizing

The Realizing relation captures whether a provided creation satisfies the requirements of another creation. It is defined between two creations of the same type, where one acts as Provider and the other as Requester. The requester specifies required properties via the Featuring relation, while the provider offers corresponding properties.

Realizing relations are inferred through rule-based matching of properties associated with creations. Separate inference rules are defined for different property types, in particular, quality attributes and physical quantities. For quality attributes, a realizing relation is inferred when a provider creation has a quality value that lies within the minimum and maximum bounds specified by the requester. These bounds are defined as features of the requester via the Featuring relation. For physical quantities, realizing is inferred analogously. A provider creation satisfies a requester if it provides a physical quantity of the same type whose value lies within the minimum and maximum bounds specified by the requester. In both cases, the provider and requester must refer to the same type of creation.

Processing

The Processing relation represents the execution of a component that transforms an input creation into an output creation. It provides a summary of a component in operation by linking the input, the executing component, and the resulting output. Processing relations are inferred based on the existence of requirement relations and previously inferred realizing relations. Each requirement associates a component with required input creations and a resulting output creation, with both creations having associated quality features.

A processing relation is inferred when an available input satisfies the requirements of a component. Each component specifies its requirements in terms of properties of an expected input creation. First, a requirement defines the properties that an input must have by associating a component with a requested creation and its features. Next, the system checks whether an available creation satisfies these required properties. This is determined through a realizing relation, which indicates that the available creation meets the constraints defined by the requested creation. If such a realizing relation exists, the requirement is considered satisfied, and a processing relation is inferred. The available creation becomes the input, the component acts as executor, and the output is defined by the requirement.

Transitive Processing

Processing relations can be transitive to represent multi-step component configurations. If the output of one processing relation serves as input to another, a new processing relation is inferred that directly links the initial input to the final output while aggregating the involved components as executors.

This transitive wrapping enables the efficient representation of multi-component configurations and binds to the Processing Requirement relation used to assess behaviors.

C. Demonstration Details

C.1 Performance Modeling

Component performance is modeled as a set of conditional mappings from environmental or input properties to output properties. Each mapping is defined over an interval of input values and assigns a fixed output value within that interval. These mappings are encoded as requirements in the ontology.

Table 1 shows the performance model of the RGB camera, where ambient light intensity determines the contrast of the produced image.

Table 1: RGB camera performance model

Light intensity (lumens)	Image contrast
0–50	0.25
50–100	0.5
100–500	0.7
500–5000	0.85

Table 2 shows the detector performance, mapping image contrast to detection probability.

Table 2: Detector performance model

Image contrast	Detection probability (p_{human})
0.1–0.4	0.25
0.4–0.6	0.5
0.6–0.8	0.7
0.8–1.0	0.8

Table 3: Tracker performance model

Env. condition	Value (m)	Component	Localization error (m)
Room size	0–2	FastTracker	2.0
Room size	2–4	FastTracker	3.0
Room size	4–6	FastTracker	4.0
Room size	6–50	FastTracker	5.0
Waypoint distance	0–2	ThoroughTracker	2.25
Waypoint distance	2–4	ThoroughTracker	3.25
Waypoint distance	4–6	ThoroughTracker	5.25
Waypoint distance	6–50	ThoroughTracker	6.25

Table 3 summarizes the performance of tracking components, where spatial properties of the environment determine localization accuracy.

Table 4 shows the performance of audio-based components.

These mappings are instantiated as requirements linking environmental conditions or input data to produced outputs. The resulting output properties (e.g., contrast, detection probability, localization error) are attached to data instances and evaluated against required constraints through realizing relations. In this way, environmental conditions directly influence which component configurations are feasible and how their performance propagates to behavior-level assessment.

C.2 Implementation Details

The demonstration is implemented in TypeDB 2.28.3 using the SEKOSA schema and instantiated data defining components, requirements, performance mappings, and environmental conditions. SEKOSA follows a hypergraph-based de-

Table 4: Audio component performance model

Env. condition	Value	Component	Output
Noise level	0–80 dB	SpeechRecognizer	$p_{human} = 0.75$
Noise level	80–90 dB	SpeechRecognizer	$p_{human} = 0.65$
Noise level	90–150 dB	SpeechRecognizer	$p_{human} = 0.3$
Room size	0–2 m	AudioTracker	2.0
Room size	2–4 m	AudioTracker	3.0
Room size	4–6 m	AudioTracker	5.0
Room size	6–50 m	AudioTracker	6.0

sign, in which relations connect multiple entities simultaneously (e.g., components, inputs, and outputs).

The knowledge base is initialized with component instances, their requirements, and performance mappings encoded as requirement instances with constrained input ranges. At runtime, environmental conditions are provided as instances of physical phenomena with concrete values and updated directly in the knowledge base.

Inference is performed using TypeDB's rule engine. Realizing relations are inferred based on satisfied requirements, enabling the inference of processing relations, which are transitively combined into component configurations. Based on these configurations, behavior-level processing requirements are inferred when available component configurations produce the desired outputs with qualities satisfying the performance criteria. As a result, behaviors that are feasible under the current conditions can be obtained directly by querying for inferred behavior instances. The presence of a behavior indicates that a corresponding component configuration exists that satisfies all required performance constraints.

TypeDB enforces consistency of the knowledge base during instantiation, updates, and inference. This work assumes the correctness of TypeDB's underlying reasoning and consistency mechanisms.